complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + reduce() aggregate (sum, max, concat)

filter() null checks + reduce() aggregate (sum, max, concat)

filter() comparisons + reduce() aggregate (sum, max, concat)

filter() method calls returning boolean + reduce() aggregate (sum, max, concat)

distinct() rely on equals() / hashcode() + reduce() aggregate (sum, max, concat)

distinct() immutable objects + reduce() aggregate (sum, max, concat)

map() transform objects + reduce() aggregate (sum, max, concat)

map() create new objects + reduce() aggregate (sum, max, concat)

map() call pure functions + reduce() aggregate (sum, max, concat)

flatmap() return stream + reduce() aggregate (sum, max, concat)

flatmap() split data + reduce() aggregate (sum, max, concat)

flatmap() expand collections + reduce() aggregate (sum, max, concat)

sorted() natural ordering + reduce() aggregate (sum, max, concat)

sorted() custom comparator + reduce() aggregate (sum, max, concat)

sorted() multi-field compare + reduce() aggregate (sum, max, concat)

peek() logging + reduce() aggregate (sum, max, concat)

peek() debugging + reduce() aggregate (sum, max, concat)

peek() metrics + reduce() aggregate (sum, max, concat)

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where the **terminal operation is reduce()** and it demonstrates **aggregation**:

- **SUM** (numeric aggregation)
- **MAX** (numeric aggregation)
- **CONCAT** (string aggregation)

---

1 filter() boolean checks + reduce() Aggregate

```
import java.util.*;

public class FilterBooleanReduceAggregate {
   public static void main(String[] args) {
      List<Integer> nums = List.of(10, 15, 20);

      int sum = nums.stream()
            .filter(n -> n % 2 == 0)
            .reduce(0, Integer::sum);

      int max = nums.stream()
            .filter(n -> n % 2 == 0)
            .reduce(Integer::max)
            .orElse(0);

      String concat = nums.stream()
            .filter(n -> n % 2 == 0)
            .map(String::valueOf)
            .reduce("", String::concat);

      System.out.println(sum + " | " + max + " | " + concat);
   }
}
```

---

2 filter() null checks + reduce() Aggregate

```
import java.util.*;

public class FilterNullReduceAggregate {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      String concat = names.stream()
            .filter(Objects::nonNull)
            .reduce("", String::concat);

      int maxLength = names.stream()
            .filter(Objects::nonNull)
            .map(String::length)
            .reduce(Integer::max)
            .orElse(0);

      System.out.println(concat + " | " + maxLength);
   }
}
```

---

3 filter() comparisons + reduce() Aggregate

```
import java.util.*;

public class FilterComparisonReduceAggregate {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      int sum = salaries.stream()
            .filter(s -> s >= 30000)
            .reduce(0, Integer::sum);

      int max = salaries.stream()
            .filter(s -> s >= 30000)
            .reduce(Integer::max)
            .orElse(0);

      System.out.println(sum + " | " + max);
   }
}
```

---

4 filter() method call boolean + reduce() Aggregate

```
import java.util.*;

public class FilterMethodReduceAggregate {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() { return active; }
   }

   public static void main(String[] args) {
      List<User> users = List.of(
            new User("Anurag", true),
            new User("Amit", true),
            new User("Ravi", false)
      );

      int count = users.stream()
            .filter(User::isActive)
            .map(u -> 1)
            .reduce(0, Integer::sum);

      String concat = users.stream()
            .filter(User::isActive)
            .map(u -> u.name)
            .reduce("", String::concat);

      System.out.println(count + " | " + concat);
   }
}
```

---

5 distinct() equals/hashCode + reduce() Aggregate

```
import java.util.*;

public class DistinctEqualsReduceAggregate {

   static class Person {
      String name;
      Person(String name) { this.name = name; }

      @Override
      public boolean equals(Object o) {
         return o instanceof Person p && name.equals(p.name);
      }

      @Override
      public int hashCode() {
         return Objects.hash(name);
      }
   }

   public static void main(String[] args) {
      List<Person> people = List.of(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      String concat = people.stream()
            .distinct()
            .map(p -> p.name)
            .reduce("", String::concat);

      System.out.println(concat);
   }
}
```

6 distinct() immutable objects + reduce() Aggregate

```
import java.util.*;

public class DistinctImmutableReduceAggregate {
   public static void main(String[] args) {
      List<String> langs = List.of("Java", "Java", "Go");

      String concat = langs.stream()
            .distinct()
            .reduce("", String::concat);

      int maxLen = langs.stream()
            .distinct()
            .map(String::length)
            .reduce(Integer::max)
            .orElse(0);

      System.out.println(concat + " | " + maxLen);
   }
}
```

---

7 map() transform objects + reduce() Aggregate

```
import java.util.*;

public class MapTransformReduceAggregate {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit");

      String concat = names.stream()
            .map(String::toUpperCase)
            .reduce("", String::concat);

      int maxLen = names.stream()
            .map(String::toUpperCase)
            .map(String::length)
            .reduce(Integer::max)
            .orElse(0);

      System.out.println(concat + " | " + maxLen);
   }
}
```

---

8 map() create new objects + reduce() Aggregate

```
import java.util.*;

public class MapNewObjectReduceAggregate {

   static class UserDTO {
      String name;
      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("Anurag");
      names.add("Amit");

      String concat = "";

      for (String name : names) {
         UserDTO dto = new UserDTO(name);
         concat += dto.name;
      }

      System.out.println(concat);
   }
}
```

---

9 map() pure functions + reduce() Aggregate

```
import java.util.*;

public class MapPureFunctionReduceAggregate {

   static int square(int n) { return n * n; }

   public static void main(String[] args) {
      List<Integer> nums = List.of(2, 3, 4);

      int sum = nums.stream()
            .map(MapPureFunctionReduceAggregate::square)
            .reduce(0, Integer::sum);

      int max = nums.stream()
            .map(MapPureFunctionReduceAggregate::square)
            .reduce(Integer::max)
            .orElse(0);

      System.out.println(sum + " | " + max);
   }
}
```

---

10 flatMap() return Stream + reduce() Aggregate

```
import java.util.*;

public class FlatMapStreamReduceAggregate {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      int sum = lists.stream()
            .flatMap(List::stream)
            .reduce(0, Integer::sum);

      System.out.println(sum);
   }
}
```

---

11 flatMap() split data + reduce() Aggregate

```
import java.util.*;

public class FlatMapSplitReduceAggregate {
   public static void main(String[] args) {
      List<String> lines = List.of("Java Stream", "Flat Map");

      String concat = lines.stream()
            .flatMap(l -> Arrays.stream(l.split(" ")))
            .reduce("", String::concat);

      System.out.println(concat);
   }
}
```

---

12 flatMap() expand collections + reduce() Aggregate

```
import java.util.*;

public class FlatMapExpandReduceAggregate {
   public static void main(String[] args) {
      Map<String, List<Integer>> map = Map.of(
            "A", List.of(1, 2),
            "B", List.of(3)
      );

      int sum = map.values().stream()
            .flatMap(Collection::stream)
            .reduce(0, Integer::sum);

      System.out.println(sum);
   }
}
```

---

13 sorted() natural ordering + reduce() Aggregate

```
import java.util.*;

public class SortedNaturalReduceAggregate {
   public static void main(String[] args) {
      List<Integer> nums = List.of(5, 1, 3);

      int max = nums.stream()
            .sorted()
            .reduce(Integer::max)
            .orElse(0);

      System.out.println(max);
   }
}
```

---

14 sorted() custom comparator + reduce() Aggregate

```
import java.util.*;

public class SortedCustomReduceAggregate {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      String concat = names.stream()
            .sorted(Comparator.reverseOrder())
            .reduce("", String::concat);

      System.out.println(concat);
   }
}
```

---

15 sorted() multi-field compare + reduce() Aggregate

```
import java.util.*;

public class SortedMultiFieldReduceAggregate {

   static class Employee {
      String name;
      int age;
      Employee(String name, int age) {
         this.name = name; this.age = age;
      }
   }

   public static void main(String[] args) {
      List<Employee> emps = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      );

      int maxAge = emps.stream()
            .sorted(Comparator
                  .comparing((Employee e) -> e.name)
                  .thenComparingInt(e -> e.age))
            .map(e -> e.age)
            .reduce(Integer::max)
            .orElse(0);

      System.out.println(maxAge);
   }
}
```

---

16 peek() logging + reduce() Aggregate

```
import java.util.*;

public class PeekLoggingReduceAggregate {
   public static void main(String[] args) {
      int sum = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Peek: " + n))
            .reduce(0, Integer::sum);

      System.out.println(sum);
   }
}
```

---

17 peek() debugging + reduce() Aggregate

```
import java.util.*;

public class PeekDebugReduceAggregate {
   public static void main(String[] args) {
      int max = List.of(10, 20, 30).stream()
            .peek(n -> {
               if (n == 20) System.out.println("Debug hit");
            })
            .reduce(Integer::max)
            .orElse(0);

      System.out.println(max);
   }
}
```

---

18 peek() metrics + reduce() Aggregate

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsReduceAggregate {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      int sum = List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .reduce(0, Integer::sum);

      System.out.println("Sum: " + sum);
      System.out.println("Processed: " + counter.get());
   }
}
```